

ed-eval-service

Technical Written Note — Bodhrik Full Stack Assessment

Candidate: Sai Pavithra G | **GitHub:** github.com/Pavithra8805/ed-eval-service |

Stack: FastAPI · PostgreSQL · Redis · Docker

1. Schema Design & Normalization Trade-offs

The schema follows **Third Normal Form (3NF)**, with four core tables: *users*, *students*, *sessions*, and *evaluations*.

users is the single source of truth for identity and role. Roles are stored as a Postgres ENUM (*admin*, *teacher*, *parent*), enforced at the DB level — not just in application code — so invalid states are impossible.

students is kept separate from users because a student is a platform entity (a child learner) but is *not* an authenticated actor. Merging students into the users table would violate single-responsibility and pollute the auth model. The student row holds a *parent_id* FK to users, which is the single join needed for parent RBAC checks.

sessions records a tutoring interaction between one teacher and one student. *teacher_id* and *student_id* are direct FKs — no junction table — because the session is inherently 1:1 in this model. If the platform later needs group sessions, a *session_participants* junction table would be the natural extension.

evaluations are intentionally separated from sessions. A single session can produce multiple evaluation runs (retry, re-grade), and each evaluation has its own lifecycle state (*pending* → *processing* → *completed/failed*). Embedding evaluation fields on the session row would preclude this and violate 2NF.

Key trade-off: UUIDs are used as primary keys throughout. This adds ~16 bytes per row versus an integer sequence, but eliminates enumerable IDs in API URLs (a security property), and makes distributed ID generation safe without coordination.

2. RBAC Evolution: Adding a Fourth Role or Nested Organisations

The current RBAC is **flat**: role logic lives in endpoint guards and query filters. This is easy to reason about with three roles, but does not scale. Here is how I would evolve it for a fourth role (e.g. *school_admin*) or nested organisations:

- **Introduce an organisations table** with a self-referential *parent_org_id* column to model nested org trees (school → district → state). Each user, student, and session would gain an *org_id* FK.

- **Move from role ENUM to a permissions table** (role → permission many-to-many). Rather than hard-coding *if role == 'teacher'* throughout the codebase, each endpoint declares which permission it requires (e.g. *sessions:read*, *evaluations:trigger*). The *require_roles* dependency becomes *require_permission*.
- **Row-level security in PostgreSQL** via RLS policies would push org-scoped filtering into the database itself, so every query automatically sees only its org's data without application-level WHERE clauses.
- **For the fourth role** (*school_admin*), add the ENUM value, create its permission set, and wire it in the dependency — no existing endpoint changes if permissions are the abstraction layer.

3. What is Missing for Production Safety

The following gaps would need to be addressed before shipping this service in production:

Migrations strategy: The lifespan handler calls *Base.metadata.create_all*, which is convenient for development but destructive in production (it silently skips columns added after initial creation). The Alembic config is already wired; the correct approach is to remove *create_all* from the app and run *alembic upgrade head* as a one-shot init container in the Kubernetes deployment or a CI/CD pipeline step before the rolling update.

Secrets handling: *SECRET_KEY*, database passwords, and Redis credentials must come from a secrets manager (AWS Secrets Manager, HashiCorp Vault, or Kubernetes Secrets sealed with Sealed Secrets / External Secrets Operator). They must never appear in *docker-compose.yml* or environment files committed to Git.

Worker reliability: The evaluation worker uses a Redis LIST as a queue. If the worker crashes mid-job, the message is lost. The production fix is to use BRPOPLPUSH (or Redis Streams with consumer groups) for at-least-once delivery, plus a dead-letter queue for jobs that exceed the retry budget.

JWT expiry & refresh tokens: The current auth issues a single access token with a configurable expiry. Production requires a short-lived access token (15 min) paired with a long-lived refresh token stored securely (HttpOnly cookie), with a token rotation endpoint and a blocklist (Redis SET) for revoked tokens.

Rate limiting, CORS hardening, and structured logging: The current CORS policy allows all origins. A production deployment should whitelist only known origins, add a rate limiter (e.g. *slowapi*), and emit structured JSON logs (via *structlog*) to a centralised log aggregation system.